

Freight Rate Prediction

Machine Learning Engineer assessment: approach, validation and December chart

Ayman Alhalabi | 02 October 2026

1. Summary

The task is to predict `posted_rate`, the price in dollars of one truck load, for 12,000 loads dated November and December 2025. The model learns from 48,000 labeled loads dated January to October 2025. Because the loads to predict lie in the future, every test in this project is a forecast test: the model is trained on earlier months and scored on later months it has never seen.

Test	Our model	Simple baseline
Sep-Oct holdout, average error in % (MAPE, clean rows)	1.64%	3.47%
Sep-Oct holdout, average error in \$ (MAE, clean rows)	\$38.97	\$77.34
Tuning folds May-Aug, mean MAPE	1.423%	4.634%
Cities never seen in training, MAPE	1.84%	3.52%

The model is XGBoost on the log of the rate per mile, on top of a small market-level layer that captures a slow upward drift in prices and a price rise in the last weeks of each quarter. The simple baseline is the typical rate per mile for the equipment type and distance band, times the distance. Clean rows exclude the 1.4% of labels that are clearly corrupted (section 3); results on all rows are reported too.

2. What the data showed

- Distance explains most of the price (correlation 0.91). Reefer loads cost about 13% more per mile than Dry Van, and Flatbed about 8% more.
- The column `quote_signal` is a trap. In January to March, June and September it is an almost exact copy of the rate per mile (correlation 0.99 or more on clean rows). In April, May, July and October it is a mirror image (about -0.99). In August, and in the November and December loads we must predict, it is pure noise. A model that uses it scores better in random testing and worse on the real task, so it is never used as a feature. For the same reason the model never sees the month.
- `market_index` is one number per day (it varies 0.025 within a day against 0.17 across days). It moves rates up and down, with a weekly cycle: highest on Thursday, lowest on Sunday and Monday.
- Beyond the index, rates drift up by about 0.6% a month, and in March, June and September they climb about 4% through the month and fall back on the 1st (figure 3). December is also a quarter-end month.
- Eight cities in the November-December file (Allentown, Charlotte, Chicago, Jackson, Knoxville, Laredo, Norfolk, San Diego) never appear in training. City effects are smooth on the map, so the model works from latitude and longitude and not from city names.

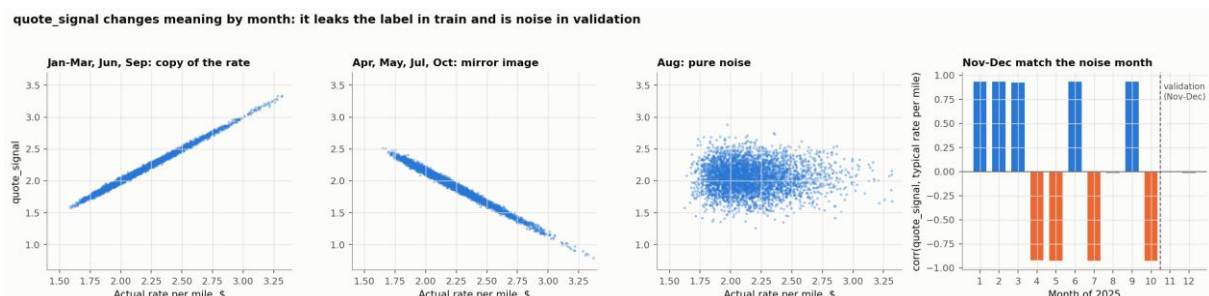


Figure 1. `quote_signal` copies the label in some months, mirrors it in others, and is noise in the months we predict.

3. Data quality issues and how they were handled

Issue	Train	Validation	What we did
Negative weight (sign flipped; same shape as positive weights)	292	145	Use the absolute value
Missing weight (missing at random)	300	165	Fill with the equipment median from the training rows
Missing market_index (missing at random)	374	249	Fill with the mean of the same date
Corrupted rate (2x to 6x too high or too low)	677	n/a	Remove from training; report scores with and without them
quote_signal leaks the label	all rows	all rows	Never used as a feature
Duplicates or spelling variants	0	0	None needed
Distance vs map distance (1.18x straight line, 70-mile floor)	consistent	consistent	Kept as given

A rate is flagged as corrupted when its rate per mile is more than 1.65 times or less than 0.61 times the typical value for its equipment and distance band. The rule is robust: every threshold from 0.3 to 0.7 on the log scale flags the same rows, because no normal rate falls in the gap. Training keeps 47,323 of 48,000 rows. Validation rows are never dropped. Every cleaning number that touches prices or weights (medians, typical rates) is learned on the training rows of each split only, so test months never leak into training. The market_index fill uses the same-date mean across both files; it reads only that input column on the same day, never a price.

4. How the data was split and validated

The submission asks for predictions on the two months after the training data ends, so a random split would be the wrong test: it lets the model see the future and hides the drift in prices. Every split here respects time.

Step	Train on	Test on	Used for
Tuning fold 1	Jan-Apr	May	Choosing the model and its settings
Tuning fold 2	Jan-May	Jun	Choosing the model and its settings
Tuning fold 3	Jan-Jun	Jul	Choosing the model and its settings
Tuning fold 4	Jan-Jul	Aug	Choosing the model and its settings
Holdout	Jan-Aug (37,944 clean rows)	Sep-Oct (9,523 rows)	One final score, after all choices were made
Unseen cities	Jan-Aug without 8 cities (33,819 rows)	Sep-Oct rows touching them (1,003)	Price of a city never seen before
Final model	Jan-Oct (47,323 clean rows)	Nov-Dec (12,000 loads)	Submission

- Two months of holdout match the two-month horizon of the real task.
- The holdout was scored once, after the model was chosen on the four tuning folds. It played no part in any choice.
- Cleaning is refit inside every fold on that fold's training rows, so no statistic from a test month reaches training.
- The eight cities held out for the unseen-city test are the eight least frequent cities (Birmingham, Dallas, Detroit, Las Vegas, Louisville, New Orleans, St. Louis, Washington). All their training rows are removed, which mimics the eight new cities in the November-December file.
- Metrics: MAE (average dollar error), MAPE (average percent error) and RMSE, each on clean rows and on all rows. The corrupted labels dominate RMSE on all rows, so model choice uses clean MAPE.

5. Model

- Target: log of the rate per mile. It validated better than the log of the rate.

- Features: distance and its log; pickup and delivery latitude and longitude; the change in latitude and longitude; the compass direction of the lane; straight-line distance and the road-to-straight-line ratio; equipment type; weight; market_index; day of the week. Never used: quote_signal, month, date, load_id. A code check stops the run if a banned column reaches the model.
- Market-level layer: a straight-line trend in time plus the quarter-end ramp, fitted by least squares on the training rows. The trees learn the rest of the price. Past the last training day the trend is extended at 0.5 of its slope, which is a cautious middle between no trend and a full trend.
- Lane correction: a small per-lane adjustment, learned out of fold and shrunk toward zero for rare lanes (prior of 10 loads). A new lane gets no correction.
- XGBoost settings: 2,000 trees, depth 5, learning rate 0.03, row and column sampling 0.8, fixed seed 42.

How each step changed the mean tuning-fold MAPE (May-Aug):

Setup	May	Jun	Jul	Aug	Mean
Simple baseline	5.13%	6.12%	3.84%	3.45%	4.634%
Ridge regression, no drift handling	1.72%	3.19%	1.83%	2.54%	2.320%
XGBoost, no drift handling	1.65%	3.19%	1.76%	2.60%	2.301%
XGBoost, recency weights	1.71%	3.13%	1.80%	1.99%	2.158%
XGBoost + trend	1.53%	2.40%	2.88%	1.80%	2.152%
XGBoost + trend + quarter-end ramp	1.57%	1.77%	1.62%	1.48%	1.610%
+ rate per mile target	1.44%	1.63%	1.53%	1.38%	1.495%
+ day of week	1.41%	1.64%	1.53%	1.37%	1.488%
+ tuned XGBoost settings	1.37%	1.61%	1.50%	1.34%	1.454%
+ lane correction (chosen)	1.34%	1.59%	1.46%	1.30%	1.423%

6. Results

Holdout: trained on January to August, scored once on September and October.

Setup	Clean MAE	Clean MAPE	Clean RMSE	Clean bias	All MAE	All MAPE	All RMSE
Chosen model	\$38.97	1.64%	\$56.50	-0.76%	\$94.98	4.16%	\$629.91
Chosen, Sep	\$36.41	1.54%	\$54.44	+0.08%	\$87.67	3.60%	\$609.93
Chosen, Oct	\$41.44	1.74%	\$58.42	-1.42%	\$102.01	4.69%	\$648.55
Simple baseline	\$77.34	3.47%	\$106.22	-0.77%	\$133.05	5.99%	\$638.01
XGBoost, no drift	\$105.80	4.42%	\$138.05	-4.27%	\$160.90	6.81%	\$648.40
Chosen without ramp	\$51.86	2.14%	\$77.59	-1.69%	\$107.70	4.63%	\$633.70

Unseen cities: on the 1,003 September-October loads that touch the eight held-out cities, the model trained without those cities scores 1.84% clean MAPE, against 1.69% when it has seen them. The cost of a new city is 0.15 percentage points, so the eight new cities in the submission file are priced almost as well as known ones.

Bias is the median signed error (below zero means predictions are too low). Results on all rows are much worse than on clean rows only because of the corrupted labels: a label that is four times too high cannot be predicted by any model. If the November-December labels contain the same 1.4% of corrupted rows, they will dominate any RMSE computed on them.

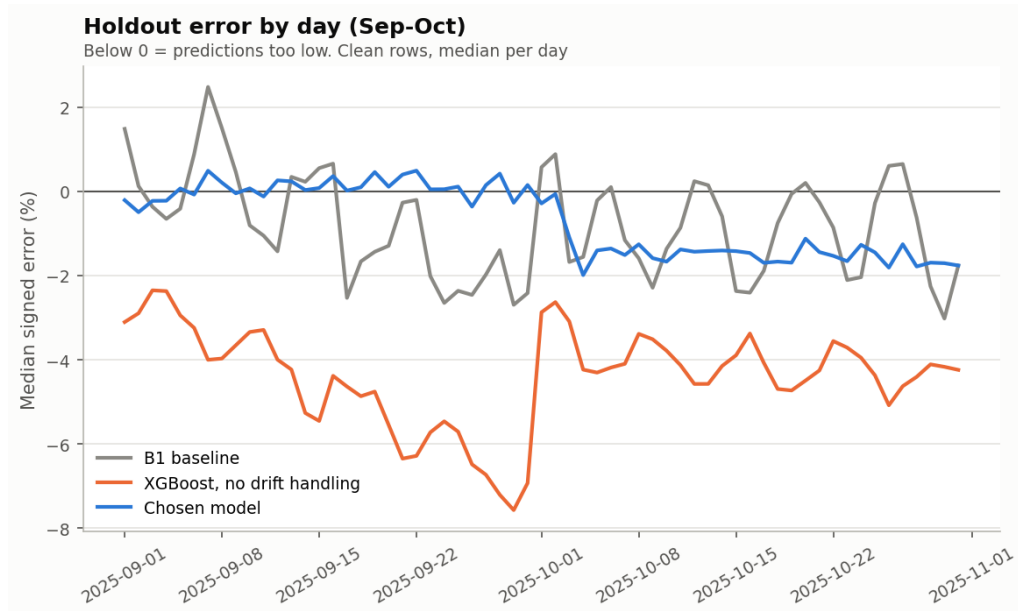


Figure 2. Daily median error on the holdout. Without drift handling XGBoost runs 2% to 8% low; the chosen model stays close to zero.

The quarter-end ramp is the largest single improvement and the biggest bet for December, so it was checked several ways. The last five days of March, June and September sit 3.1%, 3.6% and 3.4% above the trend, and no other month shows a rise. Adding the ramp raises the fit of the daily price level from R2 0.84 to 0.98, and its size is stable whichever months it is fitted on.

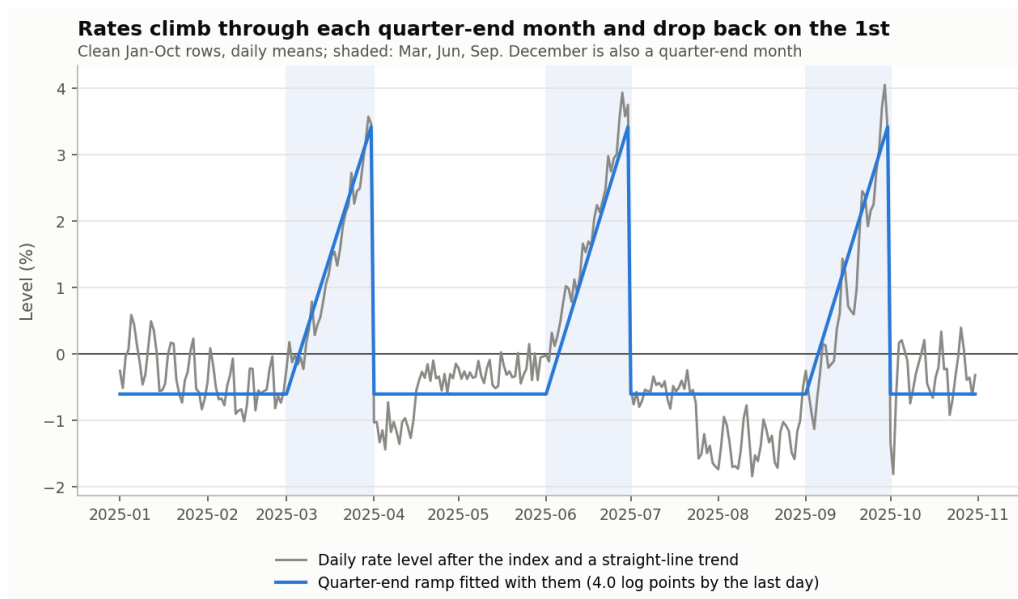


Figure 3. Daily price level after the market index and trend: a rise through each quarter-end month.

7. December prediction chart

The chart below is produced by the provided score.py from our filled december_chart_inputs.csv. Only the date changes: Lexington to Fort Wayne, 360 miles, Dry Van, 32,000 lb. The file has no map position, market_index or quote_signal, so the map positions come from the city table and market_index from the mean of the same date in the November-December file.

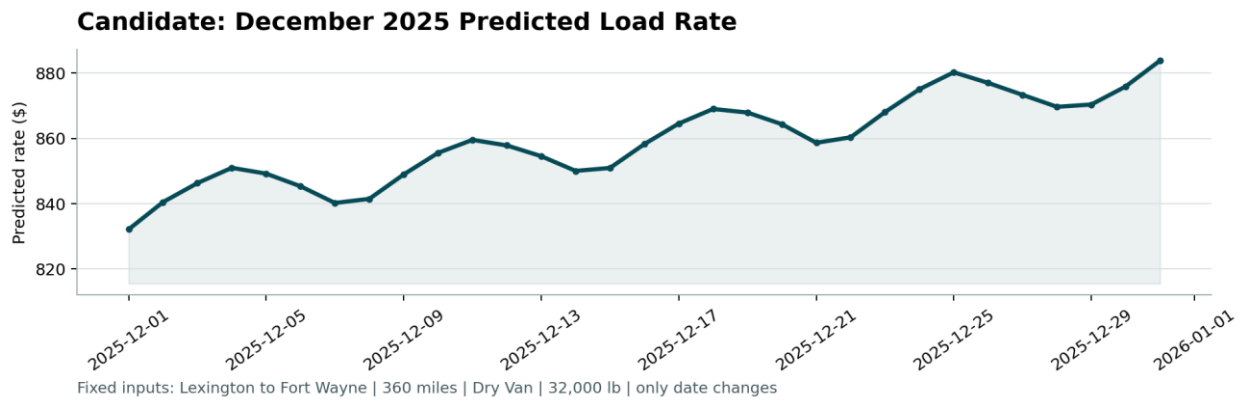


Figure 4. December 2025 predicted rate from score.py.

- Predictions run from \$832.14 to \$883.85 (mean \$859.37). All of them sit inside the range of the 21 past Dry Van loads on this lane (\$757.93 to \$934.37).
- The weekly up-and-down follows market_index: highest on Thursday, lowest on Sunday and Monday.
- The steady climb comes from the trend and the quarter-end ramp, since December ends a quarter. If December does not ramp like March, June and September, these values run about 2% high on average.

8. Quality checks

- All 93 automated checks in tests/ pass (python -m pytest -q, about 25 seconds). They cover the data contract, the cleaning counts, the time splits, leakage, both output files, the validators in score.py and an exact refit of the final model.
- An independent recount matched every headline number to six decimal places. B1 on Sep-Oct (3.47% MAPE, \$77.34 MAE) was rebuilt from the raw file with no project code, and the chosen model (1.64% MAPE, \$38.97 MAE) was refit and scored by hand.
- No banned column reaches the model. Replacing quote_signal with noise, in training or in the rows being predicted, leaves every prediction exactly the same, and so does shuffling the labels of the rows being predicted. The lane correction never uses a row's own label, and the trend and ramp are fitted on the training rows of each split only.
- A run from scratch (python run_pipeline.py, 3.5 to 7.5 minutes on our laptop) gives byte-identical prediction files, December chart and figures. Results depend slightly on the XGBoost thread count, which the code fixes at 4.
- One small defect was fixed: the memo behind the lane correction could reuse stale results when two fits shared ids and labels but had different features. It never triggered in the pipeline, and the outputs did not change. The earlier 30-minute stall was the laptop going into standby, not the code.

9. Limits and risks

- The ramp for December is learned from three quarters. It is the main source of uncertainty for the December loads.
- The trend extension is a judgment call: on the holdout, a full trend scored 1.53% and no trend 1.78%, against 1.64% for the chosen half trend. October still ran 1.4% low.
- Laredo lies south of every training city, so its prices rely on extrapolation.
- The ramp was first noticed while looking at all of January to October, so the holdout is slightly optimistic about it. The tuning folds, which use January to August only, support it on their own.

- XGBoost results depend slightly on the number of threads; the code fixes it at 4 so runs are byte-identical.

10. How to reproduce

```
python -m pip install -r requirements.txt
```

```
python run_pipeline.py
```

```
python score.py --predictions validation_predictions.csv --december-predictions  
data/december_chart_inputs.csv
```

```
python -m pytest -q
```

Data files go in data/ (see README). Every step is explained in docs/WORKLOG.md.